```
config.web_console.whitelisted_ips = %w( 127.0.0.1 192.168.0.100 )
```
Or to allow access from an entire network:

```
config.web_console.whitelisted_ips = %w( 127.0.0.1 192.168.0.0/16 )
```
The web console is a powerful tool so be careful when you give access to an IP.

4 Debugging with the byebug gem

When your code is behaving in unexpected ways, you can try printing to logs or the console to diagnose the problem. Unfortunately, there are times when this sort of error tracking is not effective in finding the root cause of a problem. When you actually need to journey into your running source code, the debugger is your best companion.

The debugger can also help you if you want to learn about the Rails source code but don't know where to start. Just debug any request to your application and use this guide to learn how to move from the code you have written deeper into Rails code.

4.1 Setup

You can use the byebug gem to set breakpoints and step through live code in Rails. To install it, just run:
```
$ gem install byebug
```
Inside any Rails application you can then invoke the debugger by calling the byebug method.
Here's an example:

```
class PeopleController < ApplicationController
  def new
    byebug
    @person = Person.new
  end
end
```

4.2 The Shell

As soon as your application calls the byebug method, the debugger will be started in a debugger shell inside the terminal window where you launched your application server, and you will be placed at the debugger's prompt (byebug). Before the prompt, the code around the line that is about to be run will be displayed and the current line will be marked by '=>'. Like this:
```
[1, 10] in /PathTo/project/app/controllers/articles_controller.rb
    3:
    4:   # GET /articles
    5:   # GET /articles.json
    6:   def index
    7:     byebug
=>  8:     @articles = Article.find_recent
    9:
   10:     respond_to do |format|
   11:       format.html # index.html.erb
   12:       format.json { render json: @articles }

(byebug)
```
If you got there by a browser request, the browser tab containing the request will be hung until the debugger has finished and the trace has finished processing the entire request.

For example: